

—— Digital Circuits ——

# 数字电路

—— ◆ ——  
学习 笔记



魏舟

2026 年 9 月 16 日

2026 秋季学期

*Caspar David Friedrich, Wanderer above the Sea of Fog, 1818*

在这里写入你的格言

---

# 目录

---

|                  |    |
|------------------|----|
| 第 1 章 数字逻辑基础     | 3  |
| 1.1 数字信号与数字电路    | 3  |
| 1.2 数制与 BCD 码    | 3  |
| 1.2.1 常用数制       | 4  |
| 十进制              | 4  |
| 二进制              | 4  |
| 十六进制和八进制         | 4  |
| 二进制与十进制之间的转换     | 5  |
| 二进制算术运算          | 5  |
| 带符号数的表示：原码、反码和补码 | 5  |
| 1.2.2 几种简单的编码    | 8  |
| 二-十进制码 (BCD 码)   | 8  |
| 格雷码 (Gray 码)     | 9  |
| 计算机字符集           | 9  |
| 1.3 逻辑代数基础       | 10 |
| 1.3.1 基本逻辑运算     | 10 |
| 1.3.2 复合逻辑运算     | 11 |
| 1.3.3 逻辑电平及正、负逻辑 | 12 |
| 1.3.4 基本定律和规则    | 13 |
| 逻辑函数的相等          | 13 |
| 基本定律             | 14 |
| 逻辑代数的三条规则        | 14 |
| 常用公式             | 16 |
| 1.3.5 逻辑函数的标准形式  | 17 |

|       |                            |    |
|-------|----------------------------|----|
|       | 函数的“与-或”式和“或-与”式 . . . . . | 17 |
|       | 最小项与最大项 . . . . .          | 17 |
|       | 逻辑函数的标准形式 . . . . .        | 19 |
|       | 真值表与逻辑表达式 . . . . .        | 20 |
| 1.3.6 | 逻辑函数的化简 . . . . .          | 22 |
|       | 公式化简法 . . . . .            | 23 |
|       | 卡诺图化简法 . . . . .           | 24 |
|       | Q-M 法（奎恩-麦克拉斯基法） . . . . . | 27 |

---

# 前言

---

本笔记按照课件《第 8 章数字逻辑基础》整理，内容涵盖数制与 BCD 码、逻辑代数基础两大部分。后续课件章节（组合逻辑电路、时序逻辑电路）将陆续补入。

整理原则：概念与公式沿用课件原文表述，仅作版式重排；课件中被提取工具打散的公式已按标准写法还原。



# 数字逻辑基础

---

## § 1.1 数字信号与数字电路

---

### 定义 1.1（模拟信号与数字信号）

- 模拟信号：在时间上和幅值上均连续的信号；
- 数字信号：在时间上和幅值上均离散的信号。

处理数字信号的电路称为**数字电路**。

**数字电路的特点：**

1. 工作信号是二进制表示的二值信号（具有“0”和“1”两种取值）；
2. 电路中器件工作于“开”和“关”两种状态，电路的输出和输入为逻辑关系；
3. 电路既能进行“代数”运算，也能进行“逻辑”运算；
4. 电路工作可靠，精度高，抗干扰性好。

## § 1.2 数制与 BCD 码

---

所谓“数制”，指进位计数制，即用进位的方法来计数。数制包括计数符号（数码）和进位规则两个方面。常用数制有十进制、十二进制、十六进制、六十进制等。

### 1.2.1 常用数制

#### 十进制

计数符号：0, 1, 2, ..., 9；进位规则：逢十进一。例如

$$1987.45 = 1 \times 10^3 + 9 \times 10^2 + 8 \times 10^1 + 7 \times 10^0 + 4 \times 10^{-1} + 5 \times 10^{-2} \quad (1.1)$$

十进制数按权展开式 ( $a_i$  为系数,  $10^i$  为权):

$$(N)_{10} = \sum_{i=-m}^{n-1} a_i \times 10^i \quad (1.2)$$

#### 二进制

计数符号：0, 1；进位规则：逢二进一。

$$(N)_2 = \sum_{i=-m}^{n-1} a_i \times 2^i \quad (1.3)$$

数字电路中采用二进制的原因：

1. 数字装置简单可靠；
2. 二进制数运算规则简单；
3. 数字电路既可以进行算术运算，也可以进行逻辑运算。

#### 十六进制和八进制

十六进制：计数符号为 0 ~ 9、A ~ F，逢十六进一：

$$(N)_{16} = \sum_{i=-m}^{n-1} a_i \times 16^i \quad (1.4)$$

例如

$$(6D.4B)_{16} = 6 \times 16^1 + D \times 16^0 + 4 \times 16^{-1} + B \times 16^{-2} = 6 \times 16 + 13 \times 16^0 + 4 \times 16^{-1} + 11 \times 16^{-2} \quad (1.5)$$

八进制：计数符号为 0 ~ 7，逢八进一：

$$(N)_8 = \sum_{i=-m}^{n-1} a_i \times 8^i \quad (1.6)$$

例如  $(63.45)_8 = 6 \times 8^1 + 3 \times 8^0 + 4 \times 8^{-1} + 5 \times 8^{-2}$ 。

## 二进制与十进制之间的转换

### (1) 二进制数转换为十进制数（按权展开法）

$$(1011.101)_2 = 1 \times 2^3 + 1 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-3} = 8 + 2 + 1 + 0.5 + 0.125 = (11.625)_{10} \quad (1.7)$$

### (2) 十进制数转换为二进制数（提取 2 的幂法）

$$\begin{aligned} (45.5)_{10} &= 32 + 8 + 4 + 1 + 0.5 \\ &= 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} \quad (1.8) \\ &= (101101.1)_2 \end{aligned}$$

数制转换还可以采用基数连乘、连除等方法。

## 二进制算术运算

算术运算规则与十进制相同，区别是逢二进一。其特点为：加、减、乘、除全部可以用**移位**和**相加**两种操作实现，电路结构简化，所以数字电路中普遍采用二进制算术运算。

**加法：**逐位相加并处理进位。

$$S_0 = A_0 + B_0 \text{ (进位 } C_1), \quad S_1 = A_1 + B_1 + C_1 \text{ (进位 } C_2), \quad \dots \quad (1.9)$$

设  $A = A_3A_2A_1A_0$ 、 $B = B_3B_2B_1B_0$ ，两数之和为  $S = S_3S_2S_1S_0$ ，则

$$A + B = C_4S_3S_2S_1S_0 \quad (1.10)$$

**减法：**将减数  $B$  变成补码后与被减数  $A$  相加，若有进位则舍去，其和即为两数之差：

$$Y = 1000 - 0100 = 1000 + (1011 + 1) = 1000 + 1100 = 10100 \xrightarrow{\text{舍去进位}} 0100 \quad (1.11)$$

**乘法：**乘以 2 相当于左移一位；方法有（1）按十进制的乘法过程；（2）采用移位相加的方法。

**除法：**除以 2 相当于右移一位。例如  $00001011 \div 010 = 00000010_B$ （商），余数为  $11_B$ 。

## 带符号数的表示：原码、反码和补码

计算机中的符号数可表示为：**符号位 + 真值**（合称机器数）。符号位用“0”表示正，用“1”表示负。

## 定义 1.2 (原码)

真值  $X$  的原码记为  $[X]_{\text{原}}$ 。在原码表示法中, 不论数的正负, 数值部分均保持原真值不变; 最高位为符号位。

**优点:** 真值和其原码表示之间的对应关系简单, 容易理解;

**缺点:** 计算机中用原码进行加减运算比较困难, **0 的表示不唯一**。8 位数 0 的原码:  $+0 = 0\ 0000000$ ,  $-0 = 1\ 0000000$ 。

## 定义 1.3 (反码)

真值  $X$  的反码记为  $[X]_{\text{反}}$ 。对一个机器数  $X$ : 若  $X > 0$ , 则  $[X]_{\text{反}} = [X]_{\text{原}}$ ; 若  $X < 0$ , 则  $[X]_{\text{反}}$  为对应原码的符号位不变、数值部分按位求反。即: **0 的反码也不唯一**:

$$[+0]_{\text{反}} = 00000000, \quad [-0]_{\text{反}} = 11111111 \quad (1.12)$$

例如  $X = -52 = -0110100$ :

$$[X]_{\text{原}} = 1\ 0110100, \quad [X]_{\text{反}} = 1\ 1001011 \quad (1.13)$$

## 定义 1.4 (补码)

真值  $X$  的补码记为  $[X]_{\text{补}}$ 。若  $X > 0$ , 则  $[X]_{\text{补}} = [X]_{\text{反}} = [X]_{\text{原}}$ ; 若  $X < 0$ , 则

$$[X]_{\text{补}} = [X]_{\text{反}} + 1 \quad (1.14)$$

例如  $X = -52 = -0110100$ :

$$[X]_{\text{原}} = 10110100, \quad [X]_{\text{反}} = 11001011, \quad [X]_{\text{补}} = [X]_{\text{反}} + 1 = 11001100 \quad (1.15)$$

**0 的补码表示唯一:**

$$[+0]_{\text{补}} = [+0]_{\text{原}} = 00000000, \quad [-0]_{\text{补}} = [-0]_{\text{反}} + 1 = 11111111 + 1 = 1\ 00000000 \quad (1.16)$$

对 8 位字长, 进位被舍掉, 故  $[-0]_{\text{补}} = 00000000$ 。

特殊数 10000000:

- 该数在原码中定义为:  $-0$ ;
- 在反码中定义为:  $-127$ ;
- 在补码中定义为:  $-128$ ;
- 对无符号数,  $(10000000)_B = 128$ 。

带符号数的表示范围 (8 位二进制数):

表 1.1: 8 位带符号数的表示范围

| 编码 | 表示范围             |
|----|------------------|
| 原码 | $-127 \sim +127$ |
| 反码 | $-127 \sim +127$ |
| 补码 | $-128 \sim +127$ |

**补码运算:** 通过引进补码, 可将减法运算转换为加法运算, 即

$$[X + Y]_{\text{补}} = [X]_{\text{补}} + [Y]_{\text{补}}, \quad [X - Y]_{\text{补}} = [X]_{\text{补}} + [-Y]_{\text{补}} \quad (1.17)$$

#### 例题 1.5 (补码加法)

$X = -0110100$ ,  $Y = +1110100$ , 求  $X + Y$ 。

解. 无论正负, 真值不变:

$$\begin{aligned} [X]_{\text{原}} &= 10110100, & [X]_{\text{补}} &= [X]_{\text{反}} + 1 = 11001100, \\ [Y]_{\text{补}} &= [Y]_{\text{原}} = 01110100 \end{aligned} \quad (1.18)$$

所以

$$[X + Y]_{\text{补}} = [X]_{\text{补}} + [Y]_{\text{补}} = 11001100 + 01110100 = 01000000 \quad (1.19)$$

即  $X + Y = (+1000000)_2 = +64$ 。

两个补码表示的二进制数相加时的符号位讨论——用二进制补码运算求出  $13 \pm 10$ 、 $-13 \pm 10$ （6 位字长）：

$$\begin{aligned}
 13 + 10 &= 001101 + 001010 = 010111 = 23, \\
 13 - 10 &= 001101 + 110110 = 000011 = 3, \\
 -13 + 10 &= 110011 + 001010 = 111101 = -3, \\
 -13 - 10 &= 110011 + 110110 = 101001 = -23
 \end{aligned} \tag{1.20}$$

无符号数的表示范围：

$$0 \leq X \leq 2^n - 1 \tag{1.21}$$

若运算结果超出这个范围，则产生**溢出**。无符号数的溢出判断准则：运算时，当**最高位向更高位有进位（或借位）**时则产生溢出。

## 1.2.2 几种简单的编码

### 二-十进制码（BCD 码）

用四位二进制代码来表示一位十进制数码，这样的代码称为二-十进制码（BCD 码，Binary Coded Decimal codes）。四位二进制有 16 种不同组合，可以在这 16 种代码中任选 10 种表示十进制数的 10 个不同符号，选择方法很多，选择方法不同就得到不同的编码形式。

表 1.2: 常用 BCD 码

| 十进制数 | 8421 码 | 5421 码 | 2421 码 | 余 3 码 |
|------|--------|--------|--------|-------|
| 0    | 0000   | 0000   | 0000   | 0011  |
| 1    | 0001   | 0001   | 0001   | 0100  |
| 2    | 0010   | 0010   | 0010   | 0101  |
| 3    | 0011   | 0011   | 0011   | 0110  |
| 4    | 0100   | 0100   | 0100   | 0111  |
| 5    | 0101   | 1000   | 1011   | 1000  |
| 6    | 0110   | 1001   | 1100   | 1001  |
| 7    | 0111   | 1010   | 1101   | 1010  |
| 8    | 1000   | 1011   | 1110   | 1011  |
| 9    | 1001   | 1100   | 1111   | 1100  |

**定义 1.6 (有权 BCD 码)**

每位数码都有确定的位权的码，例如 8421 码、5421 码、2421 码。

例：5421 码 1011 代表  $5+0+2+1=8$ ；2421 码 1100 代表  $2+4+0+0=6$ 。

5421 码和 2421 码不唯一，例如 2421 码 0110 也可表示 6。

由表 1.2 可见：

1. 8421 码和代表  $0 \sim 9$  的二进制数一一对应；
2. 5421 码的前 5 个码和 8421 码相同，后 5 个码在前 5 个码的基础上加 1000 构成；
3. 2421 码的前 5 个码和 8421 码相同，后 5 个码以中心对称取反，这样的码称为自反代码。例如  $4 \rightarrow 0100$ 、 $5 \rightarrow 1011$ ； $0 \rightarrow 0000$ 、 $9 \rightarrow 1111$ 。

**定义 1.7 (无权 BCD 码 (余 3 码))**

每位数码无确定的位权，例如余 3 码。余 3 码的编码规律为：在 8421BCD 码上加 0011。例如 6 的余 3 码为  $0110 + 0011 = 1001$ 。

**格雷码 (Gray 码)**

格雷码为无权码，特点为：相邻两个代码之间仅有一位不同，其余各位均相同，具有这种特点的代码称为循环码，格雷码是循环码。

格雷码和四位二进制码之间的关系：设四位二进制码为  $B_3B_2B_1B_0$ ，格雷码为  $R_3R_2R_1R_0$ ，则

$$R_3 = B_3, \quad R_2 = B_3 \oplus B_2, \quad R_1 = B_2 \oplus B_1, \quad R_0 = B_1 \oplus B_0 \quad (1.22)$$

其中  $\oplus$  为异或运算符，其运算规则为：若两运算数相同，结果为“0”；两运算数不同，结果为“1”。

**计算机字符集**

各种文字和符号的总称，包括各国家文字、标点符号、图形符号、数字等。常见字符集名称：

- ASCII 字符集;
- GB2312 字符集;
- BIG5 字符集;
- GB18030 字符集;
- Unicode 字符集。

## § 1.3 逻辑代数基础

数字电路的基础为逻辑代数，由英国数学家 George Boole 在 1847 年提出，逻辑代数也称布尔代数。

### 1.3.1 基本逻辑运算

在逻辑代数中，变量常用字母  $A, B, C, \dots$  表示，变量的取值只能是“0”或“1”。逻辑代数中只有三种基本逻辑运算，即“与”、“或”、“非”。

#### 定义 1.8（与逻辑运算）

只有决定一事件的全部条件都具备时，这件事才成立；如果有一个或一个以上条件不具备，则这件事就不成立。这样的因果关系称为“与”逻辑关系。其函数式为  $F = A \cdot B = AB$ 。

与门逻辑功能概括：有“0”出“0”，全“1”出“1”。

#### 定义 1.9（或逻辑运算）

在决定一事件的各种条件中，只要有一个或一个以上条件具备时，这件事就成立；只有所有的条件都不具备时，这件事才不成立。这样的因果关系称为“或”逻辑关系。其函数式为  $F = A + B$ 。

或门逻辑功能概括：有“1”出“1”，全“0”出“0”。

## 定义 1.10（非逻辑运算）

假定事件  $F$  成立与否同条件  $A$  的具备与否有关，若  $A$  具备则  $F$  不成立；若  $A$  不具备则  $F$  成立。这种因果关系称为“非”逻辑关系，其函数式为  $F = \overline{A}$ 。

表 1.3: 与、或、非逻辑真值表

| 输入  |     | 输出           |                 |                        |
|-----|-----|--------------|-----------------|------------------------|
| $A$ | $B$ | $F = AB$ （与） | $F = A + B$ （或） | $F = \overline{A}$ （非） |
| 0   | 0   | 0            | 0               | 1                      |
| 0   | 1   | 0            | 1               | 1                      |
| 1   | 0   | 0            | 1               | 0                      |
| 1   | 1   | 1            | 1               | 0                      |

注：与门和或门均可以有多个输入端。

## 1.3.2 复合逻辑运算

## 定义 1.11（与非逻辑与或非逻辑）

- 与非逻辑（将与逻辑和非逻辑组合而成）： $F = \overline{A \cdot B} = \overline{AB}$ ;
- 或非逻辑（将或逻辑和非逻辑组合而成）： $F = \overline{A + B}$ ;
- 与或非逻辑（由与、或、非三种逻辑组合而成）： $F = \overline{AB + CD}$ 。

## 定义 1.12（异或逻辑与同或逻辑）

- 异或逻辑:  $F = A\bar{B} + \bar{A}B = A \oplus B$ , 功能为相同得“0”, 相异得“1”;
- 同或逻辑:  $F = AB + \bar{A}\bar{B} = A \odot B$ , 功能为相同得“1”, 相异得“0”。

对照异或和同或逻辑真值表可以发现: 同或和异或互为反函数, 即

$$\overline{A \oplus B} = A \odot B \quad (1.23)$$

表 1.4: 复合逻辑运算真值表

| A | B | 与非 $F = \overline{AB}$ | 或非 $F = \overline{A+B}$ | 异或 $F = A \oplus B$ | 同或 $F = A \odot B$ |
|---|---|------------------------|-------------------------|---------------------|--------------------|
| 0 | 0 | 1                      | 1                       | 0                   | 1                  |
| 0 | 1 | 1                      | 0                       | 1                   | 0                  |
| 1 | 0 | 1                      | 0                       | 1                   | 0                  |
| 1 | 1 | 0                      | 0                       | 0                   | 1                  |

**注:** 教材表 8.13 给出了门电路的几种表示方法, 本课程中均采用“国标”。国外流行的电路符号常见于外文书籍中, 特别是在我国引进的一些计算机辅助分析和设计软件中常使用这些符号。

## 1.3.3 逻辑电平及正、负逻辑

门电路的输入、输出为二值信号, 用“0”和“1”表示, 这里的“0”、“1”一般用两个不同电平值来表示:

## 定义 1.13（正逻辑与负逻辑）

- 正逻辑: 用高电平  $V_H$  表示逻辑“1”, 用低电平  $V_L$  表示逻辑“0”;
- 负逻辑: 用高电平  $V_H$  表示逻辑“0”, 用低电平  $V_L$  表示逻辑“1”。

在本课程中, 如不作特殊说明, 一般都采用正逻辑表示。

$V_H$  和  $V_L$  的具体值由所使用的集成电路品种以及所加电源电压而定，有两种常用的集成电路：

1. **TTL 电路：**电源电压为 5 伏， $V_H$  约为 3 V 左右， $V_L$  约为 0.2 伏左右；
2. **CMOS 电路：**电源电压范围较宽，CMOS 4000 系列的电源电压  $V_{DD}$  为 3 ~ 18 伏；CMOS 电路的  $V_H$  约为  $0.9V_{DD}$ ，而  $V_L$  约为 0 伏左右。

对一个特定的逻辑门，采用不同的逻辑表示时，其门的名称也就不同。

表 1.5: 正负逻辑转换举例

| 电平真值表    |          |       | 正逻辑（与非门） |     |     | 负逻辑（或非门） |     |     |
|----------|----------|-------|----------|-----|-----|----------|-----|-----|
| $V_{i1}$ | $V_{i2}$ | $V_o$ | $A$      | $B$ | $Y$ | $A$      | $B$ | $Y$ |
| $V_L$    | $V_L$    | $V_H$ | 0        | 0   | 1   | 1        | 1   | 0   |
| $V_L$    | $V_H$    | $V_H$ | 0        | 1   | 1   | 1        | 0   | 0   |
| $V_H$    | $V_L$    | $V_H$ | 1        | 0   | 1   | 0        | 1   | 0   |
| $V_H$    | $V_H$    | $V_L$ | 1        | 1   | 0   | 0        | 0   | 1   |

### 1.3.4 基本定律和规则

#### 逻辑函数的相等

设有两个逻辑函数  $F_1 = f_1(A_1, A_2, \dots, A_n)$ 、 $F_2 = f_2(A_1, A_2, \dots, A_n)$ ，如果对于  $A_1, A_2, \dots, A_n$  的任何一组取值（共  $2^n$  组）， $F_1$  和  $F_2$  均相等，则称  $F_1$  和  $F_2$  相等。因此，如两个函数的真值表相等，则这两个函数一定相等。

## 基本定律

表 1.6: 逻辑代数的基本定律

| 定律    | 表达式                                        | 定律    | 表达式                                 |
|-------|--------------------------------------------|-------|-------------------------------------|
| 0-1 律 | $A \cdot 0 = 0$                            | 0-1 律 | $A + 1 = 1$                         |
| 自等律   | $A \cdot 1 = A$                            | 自等律   | $A + 0 = A$                         |
| 重迭律   | $A \cdot A = A$                            | 重迭律   | $A + A = A$                         |
| 互补律   | $A \cdot \bar{A} = 0$                      | 互补律   | $A + \bar{A} = 1$                   |
| 交换律   | $A \cdot B = B \cdot A$                    | 交换律   | $A + B = B + A$                     |
| 结合律   | $A(BC) = (AB)C$                            | 结合律   | $A + (B + C) = (A + B) + C$         |
| 分配律   | $A(B + C) = AB + AC$                       | 分配律   | $A + BC = (A + B)(A + C)$           |
| 反演律   | $\overline{A + B} = \bar{A} \cdot \bar{B}$ | 反演律   | $\overline{AB} = \bar{A} + \bar{B}$ |
| 还原律   | $\overline{\bar{A}} = A$                   |       |                                     |

反演律也称**德·摩根定理**，是一个非常有用的定理。

## 逻辑代数的三条规则

## 定义 1.14 (代入规则)

任何一个含有变量  $x$  的等式，如果将所有出现  $x$  的位置都用一个逻辑函数式  $F$  代替，则等式仍然成立。

例：已知等式  $\overline{A + B} = \bar{A} \cdot \bar{B}$ ，有函数式  $F = B + C$ ，则用  $F$  代替等式中的  $B$ ，有

$$\overline{A + (B + C)} = \bar{A} \cdot \overline{B + C} = \bar{A} \bar{B} \bar{C}, \quad \text{即} \quad \overline{A + B + C} = \bar{A} \bar{B} \bar{C} \quad (1.24)$$

由此可以证明反演定律对  $n$  个变量仍然成立。

**定义 1.15 (反演规则)**

设  $F$  为任意逻辑表达式, 若将  $F$  中所有运算符、常量及变量作如下变换:  
 $\cdot \rightarrow +$ 、 $+$   $\rightarrow \cdot$ 、 $0 \rightarrow 1$ 、 $1 \rightarrow 0$ 、原变量  $\rightarrow$  反变量、反变量  $\rightarrow$  原变量, 则  
 所得新的逻辑式即为  $F$  的反函数, 记为  $\overline{F}$ 。

**例题 1.16 (求反函数)**

解. 已知  $F = A\overline{B} + \overline{A}B$ , 根据上述规则可得

$$\overline{F} = (\overline{A} + B)(A + \overline{B}) \quad (1.25)$$

已知  $F = A + B + C + D + E$ , 则

$$\overline{F} = \overline{A}\overline{B}\overline{C}\overline{D}\overline{E} \quad (1.26)$$

由  $F$  求反函数注意: 保持原式运算的优先次序; 原式中的不属于单变量上的非号不变。

**定义 1.17 (对偶规则)**

设  $F$  为任意逻辑表达式, 若将  $F$  中所有运算符和常量作如下变换:  $\cdot \rightarrow +$ 、 $+$   $\rightarrow \cdot$ 、 $0 \rightarrow 1$ 、 $1 \rightarrow 0$ , 则所得新的逻辑表达式即为  $F$  的对偶式, 记为  $F'$ 。  
 例:  $F = AB + CD$ , 则  $F' = (A + B)(C + D)$ ;  $F = A + B + C + D + E$ , 则  $F' = ABCDE$ 。

对偶是相互的,  $F$  和  $F'$  互为对偶式。求对偶式注意: 保持原式运算的优先次序; 原式中的长短“非”号不变; 单变量的对偶式为自己。

**对偶规则:** 若有两个逻辑表达式  $F$  和  $G$  相等, 则各自的对偶式  $F'$  和  $G'$  也相等。使用对偶规则可使得某些表达式的证明更加方便, 例如:

$$A(B + C) = AB + AC \xrightarrow{\text{对偶关系}} A + BC = (A + B)(A + C) \quad (1.27)$$

## 常用公式

表 1.7: 常用公式及其证明

| 名称    | 公式                                   | 证明（对偶关系）                                                                                    |
|-------|--------------------------------------|---------------------------------------------------------------------------------------------|
| 消去律   | $AB + A\bar{B} = A$                  | $AB + A\bar{B} = A(B + \bar{B}) = A \cdot 1 = A$ ; 对偶:<br>$(A + B)(A + \bar{B}) = A$        |
| 吸收律 1 | $A + AB = A$                         | $A + AB = A(1 + B) = A \cdot 1 = A$ ; 对偶:<br>$A(A + B) = A$                                 |
| 吸收律 2 | $A + \bar{A}B = A + B$               | $A + \bar{A}B = (A + \bar{A})(A + B) = 1 \cdot (A + B) = A + B$ ; 对偶: $A(\bar{A} + B) = AB$ |
| 包含律   | $AB + \bar{A}C + BC = AB + \bar{A}C$ | 证明见式 (1.28)                                                                                 |

包含律证明:

$$\begin{aligned}
 AB + \bar{A}C + BC &= AB + \bar{A}C + (A + \bar{A})BC = AB + \bar{A}C + ABC + \bar{A}BC \\
 &= AB(1 + C) + \bar{A}C(1 + B) = AB + \bar{A}C
 \end{aligned} \tag{1.28}$$

其对偶关系为  $(A + B)(\bar{A} + C)(B + C) = (A + B)(\bar{A} + C)$ 。

**定义 1.18（异或和同或运算的性质）**

对偶数个变量而言，有

$$A_1 \oplus A_2 \oplus \cdots \oplus A_n = A_1 \odot A_2 \odot \cdots \odot A_n \tag{1.29}$$

对奇数个变量而言，有

$$A_1 \oplus A_2 \oplus \cdots \oplus A_n = \overline{A_1 \odot A_2 \odot \cdots \odot A_n} \tag{1.30}$$

异或和同或的其他性质:

表 1.8: 异或与同或运算的性质

| 异或                                              | 同或                                          |
|-------------------------------------------------|---------------------------------------------|
| $A \oplus 0 = A$                                | $A \odot 1 = A$                             |
| $A \oplus 1 = \bar{A}$                          | $A \odot 0 = \bar{A}$                       |
| $A \oplus A = 0$                                | $A \odot A = 1$                             |
| $A \oplus \bar{A} = 1$                          | $A \odot \bar{A} = 0$                       |
| $A \oplus (B \oplus C) = (A \oplus B) \oplus C$ | $A \odot (B \odot C) = (A \odot B) \odot C$ |
| $A(B \oplus C) = AB \oplus AC$                  | $A + (B \odot C) = (A + B) \odot (A + C)$   |

利用异或门可实现数字信号的极性控制；同或功能由异或门实现。

### 1.3.5 逻辑函数的标准形式

函数的“与-或”式和“或-与”式

**定义 1.19 (与-或式与或-与式)**

- “与-或”式：一个函数表达式中包含若干个“与”项，这些“与”项的“或”表示这个函数。例： $F(A, B, C, D) = \bar{A} + BC + ABCD$ ；
- “或-与”式：一个函数表达式中包含若干个“或”项，这些“或”项的“与”表示这个函数。例： $F(A, B, C, D) = (A + \bar{C} + D)(B + D)(A + \bar{B} + D)$ 。

最小项与最大项

**定义 1.20 (最小项)**

最小项是“与”项：

1.  $n$  个变量构成的每个最小项，一定是包含  $n$  个因子的乘积项；
2. 在各个最小项中，每个变量必须以原变量或反变量形式作为因子出现一次，而且仅出现一次。

例：有  $A$ 、 $B$  两变量的最小项共有四项 ( $2^2$ )： $A\bar{B}$ 、 $\bar{A}B$ 、 $\bar{A}\bar{B}$ 、 $AB$ ；有  $A$ 、 $B$ 、

$C$  三变量的最小项共有八项 ( $2^3$ )。

**最小项编号：**任一最小项用  $m_i$  表示， $m$  表示最小项，下标  $i$  为使该最小项为 1 的变量取值所对应的等效十进制数。

#### 例题 1.21 (最小项编号)

有最小项  $A\bar{B}C$ ，要使该最小项为 1， $A$ 、 $B$ 、 $C$  的取值应为 1、0、1，二进制数 101 所等效的十进制数为 5，所以

$$A\bar{B}C = m_5 \quad (1.31)$$

**最小项的性质：**

1. 变量任取一组值，仅有一个最小项为 1，其他最小项为零；
2.  $n$  变量的全体最小项之和为 1；
3. 不同的最小项相与，结果为 0；
4. 两最小项相邻，相邻最小项相“或”，可以合并成一项，并可以消去一个变量因子。

**相邻的概念：**两最小项如仅有一个变量因子不同，其他变量均相同，则称这两个最小项相邻。相邻最小项相“或”的情况：

$$\bar{A}BC + ABC = BC \quad (1.32)$$

任一  $n$  变量的最小项，必定和其他  $n$  个不同最小项相邻。

#### 定义 1.22 (最大项)

**最大项是“或”项：**

1.  $n$  个变量构成的每个最大项，一定是包含  $n$  个因子的“或”项；
2. 在各个最大项中，每个变量必须以原变量或反变量形式作为因子出现一次，而且仅出现一次。

**例：**有  $A$ 、 $B$  两变量的最大项共有四项： $A+B$ 、 $A+\bar{B}$ 、 $\bar{A}+B$ 、 $\bar{A}+\bar{B}$ ；有  $A$ 、 $B$ 、 $C$  三变量的最大项共有八项。

**最大项编号：**任一最大项用  $M_i$  表示， $M$  表示最大项，下标  $i$  为使该最大项为 0 的变量取值所对应的等效十进制数。

### 例题 1.23 (最大项编号)

有最大项  $\bar{A} + B + C$ ，要使该最大项为 0， $A$ 、 $B$ 、 $C$  的取值应为 1、0、0，二进制数 100 所等效的十进制数为 4，所以

$$\bar{A} + B + C = M_4 \quad (1.33)$$

**最大项的性质：**

1. 变量任取一组值，仅有一个最大项为 0，其它最大项为 1；
2.  $n$  变量的全体最大项之积为 0；
3. 不同的最大项相或，结果为 1；
4. 两相邻的最大项相“与”，可以合并成一项，并可以消去一个变量因子。

**相邻的概念：**两最大项如仅有一个变量因子不同，其他变量均相同，则称这两个最大项相邻。相邻最大项相“与”的情况：

$$(A + B + \bar{C})(A + B + C) = A + B \quad (1.34)$$

任一  $n$  变量的最大项，必定和其他  $n$  个不同最大项相邻。

**最小项和最大项的关系：**编号下标相同的最小项和最大项互为反函数，即

$$M_i = \overline{m_i} \quad (1.35)$$

### 逻辑函数的标准形式

**最小项之和形式**（“与或”式），例：

$$F(A, B, C) = \bar{A}\bar{B}\bar{C} + \bar{A}B\bar{C} + A\bar{B}\bar{C} = \Sigma m(2, 4, 6) \quad (1.36)$$

任一逻辑函数都可以表达为最小项之和的形式，而且是唯一的。例：

$$\begin{aligned} F(A, B, C) &= AB + \bar{A}C = AB(C + \bar{C}) + \bar{A}C(B + \bar{B}) \\ &= ABC + AB\bar{C} + \bar{A}BC + \bar{A}\bar{B}C = \Sigma m(1, 3, 6, 7) \end{aligned} \quad (1.37)$$

最大项之积形式（“或与”式），例：

$$F(A, B, C) = (A + B + C)(A + \bar{B} + C)(\bar{A} + B + C) = \Pi M(0, 2, 4) \quad (1.38)$$

任一逻辑函数都可以表达为最大项之积的形式，而且是唯一的。例：

$$\begin{aligned} F(A, B, C) &= (\bar{A} + C)(B + \bar{C}) = (\bar{A} + C + B\bar{B})(B + \bar{C} + A\bar{A}) \\ &= (\bar{A} + B + C)(\bar{A} + \bar{B} + C)(A + B + \bar{C})(\bar{A} + B + \bar{C}) = \Pi M(1, 4, 5, 6) \end{aligned} \quad (1.39)$$

最小项之和的形式和最大项之积的形式之间的关系：若  $F = \Sigma m_i$ ，则

$$\bar{F} = \sum_{j \neq i} m_j = \prod_{j \neq i} M_j \quad (1.40)$$

例：

$$F(A, B, C) = \Sigma(1, 3, 4, 6, 7) = \Pi(0, 2, 5) \quad (1.41)$$

### 真值表与逻辑表达式

真值表与逻辑表达式都是表示逻辑函数的方法。

(1) 由逻辑函数式列真值表可采用三种方法：

1. 将变量的所有取值的组合分别代入函数式，逐一算出函数值，填入真值表中；
2. 先将函数式表示为最小项之和的形式，再按最小项的性质填表；
3. 根据函数式的含义直接填表（较好，要熟练掌握）。

#### 例题 1.24（由逻辑函数式列真值表）

试列出逻辑函数式  $F(A, B, C) = AB + BC$  的真值表。

解. 方法二：先化为最小项之和的形式

$$\begin{aligned} F(A, B, C) &= AB + BC = AB(C + \bar{C}) + BC(A + \bar{A}) \\ &= ABC + AB\bar{C} + \bar{A}BC = \Sigma m(3, 6, 7) \end{aligned} \quad (1.42)$$

再根据最小项的性质，在真值表中对应于  $ABC$  取值为 011、110、111 处填“1”，其它位置填“0”，即得表 1.9。

方法三：根据函数式  $F = AB + BC$  的含义直接填表：当  $A$  和  $B$  同时为“1”（即  $AB = 1$ ）时  $F = 1$ ；当  $B$  和  $C$  同时为“1”（即  $BC = 1$ ）时  $F = 1$ ；不满足上面两种情况时  $F = 0$ 。

表 1.9:  $F(A, B, C) = AB + BC$  的真值表

| $A$ | $B$ | $C$ | $F$ |
|-----|-----|-----|-----|
| 0   | 0   | 0   | 0   |
| 0   | 0   | 1   | 0   |
| 0   | 1   | 0   | 0   |
| 0   | 1   | 1   | 1   |
| 1   | 0   | 0   | 0   |
| 1   | 0   | 1   | 0   |
| 1   | 1   | 0   | 1   |
| 1   | 1   | 1   | 1   |

**例题 1.25 (由复合函数列真值表)**

试列出  $F = (A \oplus B)(B \oplus C)$  的真值表。

解. 令  $F_1 = A \oplus B$ ,  $F_2 = B \oplus C$ ,  $F = F_1 F_2$ , 则真值表如表 1.10 所示, 同时给出了  $\overline{F}$  的值。

表 1.10:  $F = (A \oplus B)(B \oplus C)$  的真值表

| $A$ | $B$ | $C$ | $F_1 = A \oplus B$ | $F_2 = B \oplus C$ | $F = F_1 F_2$ | $\overline{F}$ |
|-----|-----|-----|--------------------|--------------------|---------------|----------------|
| 0   | 0   | 0   | 0                  | 0                  | 0             | 1              |
| 0   | 0   | 1   | 0                  | 1                  | 0             | 1              |
| 0   | 1   | 0   | 1                  | 1                  | 1             | 0              |
| 0   | 1   | 1   | 1                  | 0                  | 0             | 1              |
| 1   | 0   | 0   | 1                  | 0                  | 0             | 1              |
| 1   | 0   | 1   | 1                  | 1                  | 1             | 0              |
| 1   | 1   | 0   | 0                  | 1                  | 0             | 1              |
| 1   | 1   | 1   | 0                  | 0                  | 0             | 1              |

(2) 由真值表写逻辑函数式: 根据最小项的性质, 用观察法, 可直接从真值表写出函数的最小项之和表达式。

表 1.11: 已知真值表

| $A$ | $B$ | $C$ | $F$ |
|-----|-----|-----|-----|
| 0   | 0   | 0   | 0   |
| 0   | 0   | 1   | 0   |
| 0   | 1   | 0   | 0   |
| 0   | 1   | 1   | 1   |
| 1   | 0   | 0   | 0   |
| 1   | 0   | 1   | 1   |
| 1   | 1   | 0   | 1   |
| 1   | 1   | 1   | 1   |

**例题 1.26 (由真值表写逻辑函数式)**

由表 1.11所示的真值表求逻辑函数表达式。

解. 由真值表可见, 当  $ABC$  取 011、101、110、111 时  $F$  为“1”, 所以  $F$  由 4 个最小项组成:

$$F(A, B, C) = \Sigma m(3, 5, 6, 7) = \overline{A}BC + A\overline{B}C + AB\overline{C} + ABC \quad (1.43)$$

### 1.3.6 逻辑函数的化简

化简的意义: 节省元器件, 降低电路成本; 提高电路可靠性; 减少连线, 制作方便。

## 定义 1.27 (逻辑函数的几种常用表达式)

以  $F(A, B, C)$  为例:

$$\begin{aligned}
 F(A, B, C) &= AB + A\overline{C} && \text{与或式} \\
 &= (A + \overline{C})(A + B) && \text{或与式 (两次求对偶)} \\
 &= \overline{\overline{AB} \cdot \overline{AC}} && \text{与非——与非式 (两次求反)} \\
 &= \overline{A + \overline{C} + A + B} && \text{或非——或非式} \\
 &= \overline{\overline{AB} + \overline{AC}} && \text{与或非式}
 \end{aligned} \tag{1.44}$$

最简与或表达式的标准:

1. 所得与或表达式中, 乘积项 (与项) 数目最少;
2. 每个乘积项中所含的变量数最少。

逻辑函数常用的化简方法有公式法、卡诺图法和列表法。本课程要求掌握公式法和卡诺图法。

## 公式化简法

针对某一逻辑式, 反复运用逻辑代数公式消去多余的乘积项和每个乘积项中多余的因子, 使函数式符合最简标准。化简中常用方法如表 1.12 所示。

表 1.12: 公式化简法常用方法

| 方法   | 利用公式                                           | 例                                                                                                                                                                             |
|------|------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 并项法  | $AB + A\overline{B} = A$                       | $F = \overline{A}\overline{B}C + \overline{A}BC + A\overline{B}C + ABC = (\overline{A}\overline{B} + \overline{A}B)C + (A\overline{B} + AB)C = \overline{A}C + AC = C$        |
| 吸收法  | $A + AB = A$                                   | $F = A\overline{B} + A\overline{B}C = A\overline{B}$                                                                                                                          |
| 消项法  | $AB + \overline{A}C + BC = AB + \overline{A}C$ | $F = AB + \overline{A}C + BC = AB + \overline{A}C$                                                                                                                            |
| 消因子法 | $A + \overline{A}B = A + B$                    | $F = A + \overline{A}BC = A + BC$                                                                                                                                             |
| 配项法  | $A + \overline{A} = 1, A \cdot 1 = A$          | $F = AB + \overline{A}C + BC = AB + \overline{A}C + (A + \overline{A})BC = AB + \overline{A}C + ABC + \overline{A}BC = AB(1 + C) + \overline{A}C(1 + B) = AB + \overline{A}C$ |

**注:** 对比较复杂的函数式, 要求熟练掌握上述方法, 才能把函数化成最简。

## 卡诺图化简法

该方法是将逻辑函数用一种称为“卡诺图”的图形来表示，然后在卡诺图上进行函数化简的方法。

(1) **卡诺图的构成**：卡诺图是一种包含一些小方块的几何图形，图中每个小方块称为一个单元，每个单元对应一个最小项，两个相邻的最小项在卡诺图中也必须是相邻的。卡诺图中相邻的含义：

1. **几何相邻性**，即几何位置上相邻，也就是左右紧挨着或者上下相接；
2. **对称相邻性**，即图形中对称位置的单元是相邻的。

表 1.13: 三变量卡诺图（行列标号按循环码排列）

| $A \setminus BC$ | 00    | 01    | 11    | 10    |
|------------------|-------|-------|-------|-------|
| 0                | $m_0$ | $m_1$ | $m_3$ | $m_2$ |
| 1                | $m_4$ | $m_5$ | $m_7$ | $m_6$ |

表 1.14: 四变量卡诺图

| $AB \setminus CD$ | 00       | 01       | 11       | 10       |
|-------------------|----------|----------|----------|----------|
| 00                | $m_0$    | $m_1$    | $m_3$    | $m_2$    |
| 01                | $m_4$    | $m_5$    | $m_7$    | $m_6$    |
| 11                | $m_{12}$ | $m_{13}$ | $m_{15}$ | $m_{14}$ |
| 10                | $m_8$    | $m_9$    | $m_{11}$ | $m_{10}$ |

二、四、五变量卡诺图与之类似，行列标号均按循环码（格雷码）排列，据此判断单元之间的相邻关系。

(2) **逻辑函数的卡诺图表示法**：用卡诺图表示逻辑函数，只是把各组变量值所对应的逻辑函数  $F$  的值，填在对应的小方格中（其实卡诺图是真值表的另一种画法）。

(3) **在卡诺图上合并最小项的规则**：当卡诺图中有最小项相邻时（即有标 1 的方格相邻），可利用最小项相邻的性质对最小项合并：

- 卡诺图上任何两个标 1 的方格相邻，可以合为 1 项，并可消去 1 个变量；

- 任何四个标 1 方格相邻，可合并为一项，并可消去两个变量。四个标 1 方格相邻的特点： 同在一行或一列； 同在一田字格中；
- 任何八个标 1 的方格相邻，可以并为一项，并可消去三个变量。

综上所述，在  $n$  个变量的卡诺图中，只有  $2^i$  个相邻的标 1 方格（必须排列成方形格或矩形格的形状）才能圈在一起，合并为一项，该项保留了原来各项中  $n-i$  个相同的变量，消去  $i$  个不同变量。

#### （4）用卡诺图化简逻辑函数（化为最简与或式）

最简标准：项数最少，意味着卡诺图中圈数最少；每项中的变量数最少，意味着卡诺图中的圈尽可能大。

表 1.15:  $F(A, B, C) = \Sigma m(3, 4, 5, 6, 7)$  的卡诺图

| $A \backslash BC$ | 00 | 01 | 11 | 10 |
|-------------------|----|----|----|----|
| 0                 | 0  | 0  | 1  | 0  |
| 1                 | 1  | 1  | 1  | 1  |

#### 例题 1.28（用卡诺图化简逻辑函数）

将  $F(A, B, C) = \Sigma m(3, 4, 5, 6, 7)$  化为最简与或式，由表达式填卡诺图如表 1.15 所示。

**解.** 圈出第二行四个标 1 方格（对应变量  $A$ ）与第一行的  $m_3$ 、 $m_7$ （对应变量  $BC$ ），得最简与或式

$$F = A + BC \quad (1.45)$$

若圈法不当（如分别圈  $AB$ 、 $BC$ 、 $A\bar{B}C$  等），得  $F = AB + BC + A\bar{B}C$ ，为非最简式。

表 1.16:  $F(A, B, C, D) = \Sigma m(0, 1, 3, 7, 8, 10, 13)$  的卡诺图

| $AB \backslash CD$ | 00 | 01 | 11 | 10 |
|--------------------|----|----|----|----|
| 00                 | 1  | 1  | 1  | 0  |
| 01                 | 0  | 0  | 1  | 0  |
| 11                 | 0  | 1  | 0  | 0  |
| 10                 | 1  | 0  | 0  | 1  |

**例题 1.29 (用卡诺图化简逻辑函数 (一般步骤))**

将  $F(A, B, C, D) = \Sigma m(0, 1, 3, 7, 8, 10, 13)$  化为最简与或式, 其卡诺图如表 1.16 所示。

**解.** (1) 由表达式填卡诺图; (2) 圈出孤立的标 1 方格 ( $m_{13}$ ); (3) 找出只被一个最大的圈所覆盖的标 1 方格 ( $m_0, m_1$  等), 并圈出覆盖该标 1 方格的最大圈; (4) 将剩余的相邻标 1 方格, 圈成尽可能少、而且尽可能大的圈; (5) 将各个对应的乘积项相加, 写出最简与或式。

按上述圈法可得

$$F(A, B, C, D) = \overline{A}\overline{B}\overline{C} + \overline{A}CD + A\overline{B}\overline{D} + AB\overline{C}D \quad (1.46)$$

**(5) 化简中注意的问题:**

1. 每一个标 1 的方格必须至少被圈一次;
2. 每个圈中包含的相邻小方格数, 必须为 2 的整数次幂;
3. 为了得到尽可能大的圈, 圈与圈之间可以重叠;
4. 若某个圈中的所有标 1 方格已经完全被其它圈所覆盖, 则该圈为多余的。

**(6) 用卡诺图求反函数的最简与或式:** 在卡诺图中合并标 0 方格, 可得到反函数的最简与或式。常利用该方法求逻辑函数  $F$  的最简与或非式, 例如将  $\overline{F}$  上的非号移到右边, 就得到  $F$  的最简与或非表达式。

**(7) 复杂逻辑函数的化简:** 可将函数分解成多个部分, 先将每个部分分别填入各自的卡诺图, 然后通过卡诺图对应方格的运算, 求出函数的卡诺图, 再对卡诺图进行化简。

(8) 不完全确定的逻辑函数及其化简：在某些实际数字电路中，逻辑函数的输出只和一部分最小项有确定对应关系，而和余下的最小项无关，余下的最小项无论写入逻辑函数式还是不写入逻辑函数式，都不影响电路的逻辑功能，把这些最小项称为无关项，用英文字母  $d$  (don't care) 表示，对应的函数值记为“ $\times$ ”。包含无关项的逻辑函数称为不完全确定的逻辑函数。

#### 例题 1.30 (奇偶判别电路 (无关项的利用))

设计一个奇偶判别电路：电路输入为 8421BCD 码，当输入为偶数时输出为 0，当输入为奇数时输出为 1。

解. 由于 8421BCD 码中无 1010 ~ 1111 这 6 个码，电路禁止输入这 6 个码，这 6 个码对应的最小项为无关项，故

$$F(A, B, C, D) = \Sigma m(1, 3, 5, 7, 9) + \Sigma d(10 \sim 15) \quad (1.47)$$

若不利用无关项 (即将卡诺图中的  $\times$  均作 0 处理)，则化简结果为

$$F(A, B, C, D) = \overline{A}D + \overline{B}\overline{C}D \quad (1.48)$$

若利用无关项 (即将卡诺图中的  $\times$  按化简的需要任意处理，将有些  $\times$  当作 0，有些  $\times$  当作 1)，则化简结果为

$$F(A, B, C, D) = D, \quad \Sigma d(10 \sim 15) = 0 \quad (1.49)$$

可见利用无关项往往可以将函数化得更简单。

#### Q-M 法 (奎恩-麦克拉斯基法)

Q-M 法又称系统简化法，适用于任意多变量的函数，有较严格的算法，可借助于计算机解题。系统简化法分三个步骤进行：

1. 求出函数的全部主要项；
2. 选出函数的必要项；
3. 选择主要项，使它与必要项一起包含给定函数的全部最小项，建立函数的最简与-或式。

**注：** 在无特殊说明的情况下，为使逻辑函数化得更简单，均应按上述第二种方法（卡诺图化简）处理最小项。